initialisation, registers the address and the value to be set. The following are the steps performed by the *regmap_write* routine.

First, *regmap_write* takes the lock, which will be spinlock if *fast_io* in *regmap_config* was set; else, it will be mutex.

Next, if *max_register* is set in *regmap_config*, then it will check if the register address passed is less than *max_register*. If it is less than *max_register*, then only the write operation is performed; else, *-EIO* (invalid I/O) is returned.

After that, if the *writeable_reg* callback is set in *regmap_config*, then that callback is called. If that callback returns 'true', then further operations are done; if it returns 'false', then an error *-EIO* is returned. This step is only performed if *writeable_reg* is set.

If *writeable_reg* is not set, but *wr_table* is set, then there's a check on whether the register address lies in *no_ranges*, in which case an *-EIO* error is returned; else, it is checked whether it lies in the *yes_ranges*. If it is not present there, then an *-EIO* error is returned and the operation is terminated. If it lies in the *yes_ranges*, then further operations are performed. This step is only performed if *wr_table* is set; else, it is skipped.

Whether caching is permitted is now checked. If it is permitted, then the register value is cached instead of writing directly to hardware, and the operation finishes at this step. If caching is not permitted, it goes to the next step. Caching will be discussed later.

After this hardware write routine is called to write the value in the hardware register, this function writes the *write_flag_mask* to the first byte of the value and the value is written to the device.

After completing the write operation, the lock that was taken before writing is released and the function returns.

regmap_read: This function is used to read data from the device. It takes in regmap structure returned during initialisation, and registers the address and value pointer in which data is to be read. The following steps are performed during register reading.

First, the read function will take a lock before performing the read operation. This will be a spinlock if *fast_io* is set in *regmap_config;* else, regmap will use mutex.

Next, it will check whether the passed register address is less than *max_register;* if it is not, then *-EIO* is returned. This step is only done if *max_register* is set greater than zero.

Then, it will check if the *readable_reg* callback is set. If it is, then that callback is called, and if this callback returns 'false' then the read operation is terminated returning an *-EIO* error. If this callback returns 'true' then further operations are performed.

Note: This step is only performed if *readable_reg* is set.

What is checked next is whether the register address lies in the *no_ranges* of the *rd_table* in config. If it does, then an *-EIO* error is returned. If it doesn't lie either in the *no_ranges* or in the *yes_ranges*, then too an *-EIO* error is returned.

Only if it lies in the *yes_ranges* can further operations be performed. This step is only performed if the *rd_table* is set.

Now, if caching is permitted, then the register value is read from the cache and the function returns the value being read. If caching is set to bypass, then the next step is performed.

After the above steps have been taken, the hardware read operation is called to read the register value, and the value of the variable which was passed is updated with the value returned.

The lock that was taken before starting this operation is now released and the function returns.

Writing a regmap based driver

Let us try to write a driver based on the regmap framework.

Let us assume that there is a device X connected on an SPI bus, which has the following properties:

- 8-bit register address
- 8-bit register values
- 0x80 as write mask
- It's a fast I/O device so the spinlock should be used
- Valid address range:
  - 0x20 to 0x4F
  - 0x60 to 0x7F

The driver can be written as follows:

```
//include other include files
#include <linux/regmap.h>

static struct custom_drv_private_struct
{
            //other fields relevant to device
            struct regmap *map;
};

static const struct regmap_range wr_rd_range[] =
{
            {
                        .range_min = 0x20,
                        .range_max = 0x4F,
            }, {
                        .range_min = 0x60,
                        .range_max = 0x7F
            },
};
struct regmap_access_table drv_wr_table =
{
             .yes_ranges =   wr_rd_range,
.n_yes_ranges = ARRAY_SIZE(wr_rd_range),
};

struct regmap_access_table drv_rd_table =
{
         .yes_ranges =   wr_rd_range,
         .n_yes_ranges = ARRAY_SIZE(wr_rd_range),
```